

DOCUMENTACIÓN ENDPOINTS

ÍNDICE DE CONTENIDOS

1. Gestión de Empresas (EmpresaController) [Pág. 2](#)
(Endpoints de consulta, filtros por centro/alumno y guía de escritura)
2. Gestión de Centros Docentes (CentroDocenteController) [Pág. 4](#)
(Acceso al centro principal, detalle por código y gestión de entidades)
3. Gestión de Asistencias (AsistenciaController) [Pág. 6](#)
(Registro de entrada/salida, historial y cálculo automático de horas)
4. Gestión de Tareas (TareaController) [Pág. 8](#)
(Flujo de estados: Creada, Asignada, Completada, Validada y filtros)
5. Gestión de Personal Docente (ProfesorController) [Pág. 11](#)
(Administración de Gestores y Tutores por centro y rol)
6. Gestión de Tutores de Empresa (TutorEmpresaController) [Pág. 13](#)
(Vinculación de tutores a empresas y seguimiento de alumnos asignados)
7. Gestión de Alumnos (AlumnoController) [Pág. 15](#)
(Administración de matrículas, filtro de huérfanos y asignación de tutores)

NOTA DE PRERREQUISITOS PARA PRUEBAS Para la correcta ejecución de los enlaces y pruebas descritos en este documento, es imprescindible que:

1. El motor de base de datos esté activo y con los datos de prueba cargados.
2. La aplicación Spring Boot esté corriendo en el puerto **8088**.
3. Se respete la secuencia de pasos en los módulos de escritura para mantener la integridad de los datos.
4. Puesta en Marcha: En caso de dudas sobre la configuración del entorno o el despliegue inicial, consultar el archivo README.md de la raíz del proyecto donde encontrará la documentación para poner en marcha ambas cosas.

1. Módulo: Gestión de Empresas (EmpresaController)

Base URL: <http://localhost:8088/api/empresas>

Método	Endpoint	Propósito en el Front-end (Interfaz)
GET	/centro/{cod}	Rellenar el listado de empresas cuando el Profesor selecciona su Centro.
GET	/alumno/{id}	Mostrar los datos de la empresa en el perfil del Alumno.
POST	/	Formulario de "Alta de Nueva Empresa" del Gestor.
PUT	/cif}	Pantalla de "Edición de Empresa" para corregir datos.

Métodos, se pueden probar en el navegador web:

Método	Acción	Enlace Completo de Prueba (Puerto 8088)	Resultado Esperado
GET	Detalle CIF	http://localhost:8088/api/empresas/CIF001	Datos de <i>Tech Solutions S.L.</i>
GET	Buscar Nombre	http://localhost:8088/api/empresas/buscar?nombre=Software	Lista con <i>Andalucía Software</i>
GET	Por Centro	http://localhost:8088/api/empresas/centro/CEN02	Empresas en Sevilla (CIF003, 004, 006)
GET	Por Alumno	http://localhost:8088/api/empresas/alumno/ALU05	Empresa <i>Data Mining Corp</i> (CIF005)

Guía de Pruebas de Escritura (POST, PUT, DELETE)

[IMPORTANTE]

Herramienta Recomendada: Para ejecutar estas pruebas, se requiere el uso de Postman, Insomnia o la extensión Thunder Client de VS Code. El navegador estándar solo permite realizar peticiones GET.

Para asegurar el correcto funcionamiento y la integridad de los datos, se recomienda seguir estrictamente esta secuencia de tres pasos:

PASO 1: Creación de Entidad (POST) Utilizaremos este JSON para dar de alta una nueva empresa en el sistema.

URL: `http://localhost:8088/api/empresas`

Método: POST

Body (JSON):

```
{
  "cif": "CIF-9999",
  "razonSocial": "Empresa de Pruebas BETA",
  "direccion": "Avenida de la Tecnología 45",
  "localidad": "Sevilla",
  "emailContacto": "testing@beta.es"
}
```

Resultado esperado: Status 201 Created y el objeto devuelto con los datos confirmados.

PASO 2: Modificación de los datos creados (PUT) Una vez creada la empresa CIF-9999, procederemos a actualizar su razón social para verificar el endpoint de edición.

URL: `http://localhost:8088/api/empresas/CIF-9999`

Método: PUT

Body (JSON):

```
{
  "cif": "CIF-9999",
  "razonSocial": "Empresa de Pruebas ACTUALIZADA",
  "direccion": "Avenida de la Tecnología 45",
  "localidad": "Sevilla",
  "emailContacto": "soporte@beta.es"
}
```

Resultado esperado: Status 200 OK y la razón social actualizada a "ACTUALIZADA".

PASO 3: Limpieza del sistema (DELETE) Para finalizar la secuencia y no dejar datos de prueba residuales, eliminamos la entidad creada.

URL: `http://localhost:8088/api/empresas/CIF-9999`

Método: DELETE

Resultado esperado: Status 204 No Content. Si intentas hacer un GET posterior a ese CIF, el sistema debería devolver un 404 Not Found.

2. Módulo: Gestión de Centros Docentes (CentroDocenteController)

Base URL: <http://localhost:8088/api/centros>

Método	Endpoint	Propósito en el Front-end (Interfaz)
GET	/principal	Cargar el logo y nombre del centro en la cabecera (Header) de la App.
GET	//{codigo}	Pantalla de "Ajustes del Centro" para visualizar la configuración institucional.
GET	/alumno/{id}	Mostrar el centro educativo de referencia en el carnet o ficha del alumno.
POST	/	Formulario de configuración inicial del sistema (Solo perfil Gestor).
PUT	//{codigo}	Pantalla de edición para actualizar datos de contacto o dirección del centro.
DELETE	//{id}	Opción de borrado técnico de la entidad (Sujeto a restricciones).

Métodos, se pueden probar en el navegador web:

Método	Acción	Enlace Completo de Prueba (Puerto 8088)	Resultado Esperado
GET	Detalle Centro	http://localhost:8088/api/centros/CEN01	Datos de <i>IES Tecnológico Madrid</i> .
GET	Centro Alumno	http://localhost:8088/api/centros/alumno/ALU11	Datos de <i>IES Innovación Sevilla</i> .
GET	Acceso Principal	http://localhost:8088/api/centros/principal	Datos del centro configurado como principal.

Guía de Pruebas de Escritura (POST, PUT, DELETE)

[IMPORTANTE]

Herramienta Recomendada: Para ejecutar estas pruebas, se requiere el uso de Postman, Insomnia o la extensión Thunder Client de VS Code. El navegador estándar solo permite realizar peticiones GET.

Para asegurar el correcto funcionamiento y la integridad de los datos, se recomienda seguir estrictamente esta secuencia de tres pasos:

PASO 1: Creación de Entidad (POST) Utilizaremos este JSON para dar de alta un nuevo centro docente en el sistema.

URL: `http://localhost:8088/api/centros`

Método: POST

Body (JSON):

```
{
  "codigoCentro": "CEN-TEST",
  "nombreOficial": "Centro de Pruebas Unitarias",
  "direccion": "Calle del Código 1",
  "localidad": "Málaga",
  "telefono": "952000000",
  "correoInstitucional": "admin@centroprebas.es"
}
```

Resultado esperado: Status 201 Created y el objeto devuelto con los datos confirmados.

PASO 2: Modificación de los datos creados (PUT) Una vez creado el centro CEN-TEST, procederemos a actualizar su nombre oficial y teléfono.

URL: `http://localhost:8088/api/centros/CEN-TEST`

Método: PUT

Body (JSON):

```
{
  "codigoCentro": "CEN-TEST",
  "nombreOficial": "Centro de Pruebas ACTUALIZADO",
  "direccion": "Calle del Código 1",
  "localidad": "Málaga",
  "telefono": "952111222",
  "correoInstitucional": "soporte@centroprebas.es"
}
```

Resultado esperado: Status 200 OK y el nombre actualizado a "ACTUALIZADO".

PASO 3: Limpieza del sistema (DELETE) Para finalizar la secuencia y no dejar datos de prueba residuales, eliminamos la entidad creada.

URL: `http://localhost:8088/api/centros/CEN-TEST`

Método: DELETE

Resultado esperado: Status 204 No Content.

3. Módulo: Gestión de Asistencias (AsistenciaController)

Base URL: <http://localhost:8088/api/asistencias>

Método	Endpoint	Propósito en el Front-end (Interfaz)
POST	/entrada/{idAlumno}	Botón "Registrar Entrada" en el Dashboard del Alumno.
PATCH	/salida/{idAlumno}	Botón "Finalizar Jornada" con cuadro de texto para observaciones.
GET	/buscar	Consultar si el alumno ya ha fichado en el día actual al cargar la App.
GET	/historial/{id}	Visualizar el calendario o tabla de asistencias pasadas del alumno.

Métodos, se pueden probar en el navegador web:

Método	Acción	Enlace Completo de Prueba (Puerto 8088)	Resultado Esperado
GET	Buscar por Fecha	http://localhost:8088/api/asistencias/buscar?idAlumno=ALU01&fecha=2026-03-16	Registro de asistencia con "Configuración de entorno Docker".
GET	Historial Alumno	http://localhost:8088/api/asistencias/historial/ALU11	Lista de las 4 asistencias cargadas en el SQL para el alumno Once.

Guía de Pruebas de Escritura (POST, PUT, DELETE)

[IMPORTANTE]

Herramienta Recomendada: Para ejecutar estas pruebas, se requiere el uso de Postman, Insomnia o la extensión Thunder Client de VS Code. El navegador estándar solo permite realizar peticiones GET.

Para asegurar el correcto funcionamiento y la integridad de los datos, se recomienda seguir estrictamente esta secuencia de tres pasos:

PASO 1: Registro de Entrada (POST) Simula que el alumno inicia su jornada.

URL: `http://localhost:8088/api/asistencias/entrada/ALU20`

Método: POST

Parámetros (Opcional): `?hora=08:00:00` (Si no se envía, el sistema toma la hora actual).

Resultado esperado: Status 200 OK. Se crea un registro para el alumno Veinte con hora_entrada y fecha actual.

PASO 2: Registro de Salida y Cierre (PATCH) Simula que el alumno termina su jornada y añade lo que ha hecho.

URL: `http://localhost:8088/api/asistencias/salida/ALU20?hora=14:30:00`

Método: PATCH

Body (Texto plano / String): ```text`

Finalización de las pruebas unitarias y subida al repositorio.

Resultado esperado:

```
{  
  "id": 25,  
  "fecha": "2026-03-23",  
  "horaEntrada": "08:00:00",  
  "horaSalida": "14:30:00",  
  "horasDiarias": 6.5,  
  "observaciones": "Finalización de las pruebas unitarias y subida al repositorio.",  
  "fkAlumno": "ALU20"  
}
```

4. Módulo: Gestión de Tareas (TareaController)

Base URL: <http://localhost:8088/api/tareas>

Método	Endpoint	Propósito en el Front-end (Interfaz)
POST	/	Formulario del Tutor de Empresa para crear una nueva tarea (Borrador).
PUT	/id/asignar	Botón "Asignar" para que la tarea aparezca en el panel del alumno.
PATCH	/id/estado	El Alumno marca la tarea como "COMPLETADA" e introduce las horas reales.
PATCH	/id/revisar	El Profesor valida la tarea tras supervisarla (Estado "VALIDADA").
GET	/alumno/{id}	Cargar el listado de tareas (Kanban o Tabla) en el perfil del Alumno.
GET	/tutor/{id}/estado/{e}	Filtro para que el Tutor vea qué tareas están pendientes o terminadas.

Métodos, se pueden probar en el navegador web:

Método	Acción	Enlace Completo de Prueba (Puerto 8088)	Resultado Esperado
GET	Tareas Alumno	http://localhost:8088/api/tareas/alumno/ALU01	Array JSON con las 4 tareas de ALU01 (Docker, CSS, etc.).
GET	Filtro Tutor	http://localhost:8088/api/tareas/tutor/TUT001/estado/ASIGNADA	Lista de tareas asignadas por el Tutor Carlos que aún no se han terminado.

Guía de Pruebas de Escritura (POST, PUT, PATCH, DELETE) [IMPORTANTE]

Herramienta Recomendada: Para ejecutar estas pruebas, se requiere el uso de Postman, Insomnia o la extensión Thunder Client de VS Code. El navegador estándar solo permite realizar peticiones GET.

Para asegurar el correcto funcionamiento y la integridad de los datos, se recomienda seguir estrictamente esta secuencia de tres pasos:

PASO 1: Creación de Tarea (POST) El Tutor crea la tarea en estado inicial.

URL: <http://localhost:8088/api/tareas>

Método: POST

Body (JSON):

```
{ "titulo": "Implementación de Unit Tests",  
  "descripcion": "Crear pruebas para el servicio de autenticación usando JUnit",  
  "fechaLimite": "2026-03-30",  
  "horasEstimadasla": 4.5,  
  "fkTutorEmpresa": "TUT001",  
  "fkAlumno": "ALU01",  
  "fkProfesorTutor": "DOC002" }
```

Resultado esperado:

```
{ "id": 25,  
  "titulo": "Implementación de Unit Tests",  
  "estado": "CREADA",  
  "fkAlumno": "ALU01" }
```

PASO 2: Asignación y Activación (PUT) Se activa la tarea para que el alumno la reciba oficialmente.

URL: <http://localhost:8088/api/tareas/25/asignar>

Método: PUT

Resultado esperado:

```
{ "id": 25,  
  "titulo": "Implementación de Unit Tests",  
  "estado": "ASIGNADA",  
  "fechaAsignacion": "2026-03-23" }
```

PASO 3: Finalización por parte del Alumno (PATCH) El alumno termina el trabajo e indica las horas reales dedicadas (RN-05).

URL: <http://localhost:8088/api/tareas/25/estado?nuevoEstado=COMPLETADA&horas=5.0>

Método: PATCH

Resultado esperado:

```
{ "id": 25,  
  "estado": "COMPLETADA",  
  "horasReales": 5.0 }
```

PASO 4: Revisión del Profesor (PATCH)

El profesor supervisa y cierra el ciclo de la tarea.

URL: <http://localhost:8088/api/tareas/25/revisar>

Método: PATCH

Resultado esperado:

```
{ "id": 25,  
  "estado": "VALIDADA" }
```

PASO 5: Limpieza (DELETE)

Eliminación de la tarea de prueba.

URL: <http://localhost:8088/api/tareas/25>

Método: DELETE

Resultado esperado: Status 204 No Content.

5. Módulo: Gestión de Personal Docente (ProfesorController)

Base URL: <http://localhost:8088/api/profesores>

Método	Endpoint	Propósito en el Front-end (Interfaz)
POST	/	Formulario de alta para nuevos Profesores o Gestores del centro.
GET	/ {id}	Cargar el perfil del docente logueado o ver detalles de un profesor.
GET	/centro/{cod}	Listar el claustro de un centro (ej: para asignar tutores a alumnos).
GET	/alumno/{id}	Mostrar quién es el profesor tutor responsable de un alumno concreto.
PUT	/ {id}	Pantalla de edición para actualizar el email o rol del docente.
DELETE	/ {id}	Baja de un docente del sistema (Sujeto a integridad referencial).

Métodos, se pueden probar en el navegador web:

Método	Acción	Enlace Completo de Prueba (Puerto 8088)	Resultado Esperado
GET	Detalle Docente	http://localhost:8088/api/profesores/DOC001	Datos de Juan El Gestor (Rol: GESTOR).
GET	Listar Equipo	http://localhost:8088/api/profesores/centro/CEN02?rol=TUTOR	Lista de los 4 tutores de Sevilla (DOC005 al 008).
GET	Tutor de Alumno	http://localhost:8088/api/profesores/alumno/ALU01	Datos de Ana García (DOC002), tutora de ALU01.

Guía de Pruebas de Escritura (POST, PUT, DELETE) [IMPORTANTE]

Herramienta Recomendada: Para ejecutar estas pruebas, se requiere el uso de Postman, Insomnia o la extensión Thunder Client de VS Code. El navegador estándar solo permite realizar peticiones GET.

Para asegurar el correcto funcionamiento y la integridad de los datos, se recomienda seguir estrictamente esta secuencia de tres pasos:

PASO 1: Creación de Entidad (POST) Utilizaremos este JSON para dar de alta un nuevo profesor en un centro existente.

URL: `http://localhost:8088/api/profesores`

Método: POST

Body (JSON):

```
{ "codigoDocente": "DOC-999",  
  "nombre": "Nacho",  
  "apellidos": "Pruebas",  
  "email": "nacho.test@iesmadrid.es",  
  "rol": "TUTOR",  
  "fkCentro": "CEN01" }
```

Resultado esperado:

```
{ "codigoDocente": "DOC-999",  
  "nombre": "Nacho",  
  "apellidos": "Pruebas",  
  "email": "nacho.test@iesmadrid.es",  
  "rol": "TUTOR",  
  "fkCentro": "CEN01" }
```

PASO 2: Modificación de los datos creados (PUT) Actualizamos el rol del docente recién creado a GESTOR.

URL: `http://localhost:8088/api/profesores/DOC-999`

Método: PUT

Body (JSON):

```
{ "codigoDocente": "DOC-999",  
  "nombre": "Nacho",  
  "apellidos": "Pruebas",  
  "email": "nacho.gestor@iesmadrid.es",  
  "rol": "GESTOR",  
  "fkCentro": "CEN01" }
```

Resultado esperado:

```
{ "codigoDocente": "DOC-999",  
  "rol": "GESTOR",  
  "email": "nacho.gestor@iesmadrid.es" }
```

PASO 3: Limpieza del sistema (DELETE) Eliminamos el registro de prueba creado.

URL: `http://localhost:8088/api/profesores/DOC-999`

Método: DELETE

Resultado esperado: Status 204 No Content. Si intentas borrar un profesor con alumnos asignados (como DOC002), el sistema devolverá un error de integridad (RN-03).

6. Módulo: Gestión de Tutores de Empresa (TutorEmpresaController)

Base URL: <http://localhost:8088/api/tutores-empresa>

Método	Endpoint	Propósito en el Front-end (Interfaz)
POST	/empresa/{cif}	Formulario para dar de alta un tutor dentro de una empresa específica.
GET	/idTutor/perfil	Cargar los datos personales del tutor en su panel de configuración.
GET	/empresa/{cif}	Listado de tutores disponibles para asignar a un alumno en una empresa.
GET	/idTutor/alumnos	Visualizar la lista de alumnos asignados bajo la supervisión del tutor.
PUT	/idTutor	Pantalla de edición para actualizar el contacto o cargo del tutor.
DELETE	/idTutor	Eliminar un tutor del sistema (Sujeto a integridad referencial).

Métodos, se pueden probar en el navegador web:

Método	Acción	Enlace Completo de Prueba (Puerto 8088)	Resultado Esperado
GET	Perfil Tutor	http://localhost:8088/api/tutores-empresa/TUT001/perfil	Datos de Carlos Empresa1 (Vinculado a CIF001).
GET	Tutores Empresa	http://localhost:8088/api/tutores-empresa/empresa/CIF002	Lista de tutores de Sistemas Avanzados SA (TUT002).
GET	Mis Alumnos	http://localhost:8088/api/tutores-empresa/TUT001/alumnos	Lista con ALU01, ALU02, ALU07 y ALU10 (Asignados a Carlos).

Guía de Pruebas de Escritura (POST, PUT, DELETE)
[IMPORTANTE]

Herramienta Recomendada: Para ejecutar estas pruebas, se requiere el uso de Postman, Insomnia o la extensión Thunder Client de VS Code. El navegador estándar solo permite realizar peticiones GET.

Para asegurar el correcto funcionamiento y la integridad de los datos, se recomienda seguir estrictamente esta secuencia de tres pasos:

PASO 1: Creación de Entidad (POST) Utilizaremos este JSON para dar de alta un nuevo tutor en la empresa CIF006.

URL: `http://localhost:8088/api/tutores-empresa/empresa/CIF006`

Método: POST

Body (JSON):

```
{ "codigoTutor": "TUT-TEST",  
  "nombre": "Marta",  
  "apellidos": "Odoo Experta",  
  "email": "marta.odoo@empresa.es" }
```

Resultado esperado:

```
{ "codigoTutor": "TUT-TEST",  
  "nombre": "Marta",  
  "apellidos": "Odoo Experta",  
  "email": "marta.odoo@empresa.es",  
  "fkEmpresa": "CIF006" }
```

PASO 2: Modificación de los datos creados (PUT) Actualizamos los apellidos del tutor recién creado.

URL: `http://localhost:8088/api/tutores-empresa/TUT-TEST`

Método: PUT

Body (JSON):

```
{ "codigoTutor": "TUT-TEST",  
  "nombre": "Marta",  
  "apellidos": "Odoo Senior",  
  "email": "marta.senior@empresa.es" }
```

Resultado esperado:

```
{ "codigoTutor": "TUT-TEST",  
  "apellidos": "Odoo Senior",  
  "email": "marta.senior@empresa.es" }
```

PASO 3: Limpieza del sistema (DELETE) Eliminamos el registro de prueba creado.

URL: `http://localhost:8088/api/tutores-empresa/TUT-TEST`

Método: DELETE

Resultado esperado: Status 204 No Content. Si intentas borrar un tutor que tiene alumnos asignados (como TUT001), el sistema devolverá un error de integridad (RN-03).

7. Módulo: Gestión de Alumnos (AlumnoController)

Base URL: <http://localhost:8088/api/alumnos>

Método	Endpoint	Propósito en el Front-end (Interfaz)
POST	/	Formulario de alta de nuevos alumnos (Rol Gestor).
GET	//{id}	Cargar el perfil del alumno, sus datos de contacto y centro.
GET	/centro/{cod}	Listar todos los alumnos matriculados en un centro específico.
GET	/huerfanos	Panel del Gestor para detectar alumnos sin tutor docente asignado.
GET	/tutor/{id}	Listado de alumnos que supervisa un profesor concreto.
GET	/empresa/{cif}	Listado de alumnos que están realizando las FCT en una empresa.
PUT	//{id}	Edición de la ficha del alumno (cambio de empresa, tutor o contacto).
DELETE	//{id}	Baja de un alumno del sistema.

Métodos, se pueden probar en el navegador web:

Método	Acción	Enlace Completo de Prueba (Puerto 8088)	Resultado Esperado
GET	Perfil Alumno	http://localhost:8088/api/alumnos/ALU01	Datos de Alumno Uno (IES Tecnológico Madrid).
GET	Por Centro	http://localhost:8088/api/alumnos/centro/CEN02	Lista de los 10 alumnos de Sevilla (ALU11 al ALU20).
GET	Listar Huérfanos	http://localhost:8088/api/alumnos/huerfanos	Lista de alumnos que no tienen fk_profesor asignado (Vacío).
GET	Por Empresa	http://localhost:8088/api/alumnos/empresa/CIF001	Lista de alumnos en Tech Solutions (ALU01, 02, 07, 10).

Guía de Pruebas de Escritura (POST, PUT, DELETE)

[IMPORTANTE]

Herramienta Recomendada: Para ejecutar estas pruebas, se requiere el uso de Postman, Insomnia o la extensión Thunder Client de VS Code. El navegador estándar solo permite realizar peticiones GET.

Para asegurar el correcto funcionamiento y la integridad de los datos, se recomienda seguir estrictamente esta secuencia de tres pasos:

PASO 1: Creación de Entidad (POST) Damos de alta a un nuevo alumno. El servicio asignará la contraseña por defecto "1234" si no se envía una.

URL: `http://localhost:8088/api/alumnos`

Método: POST

Body (JSON):

```
{ "idCodigoAlumno": "ALU-TEST",  
  "nombre": "Jorge",  
  "apellidos": "Pruebas Finales",  
  "email": "jorge.test@gmail.com",  
  "fkCentro": "CEN01",  
  "fkEmpresa": "CIF001",  
  "fkProfesor": "DOC002",  
  "fkTutor": "TUT001" }
```

Resultado esperado:

```
{ "idCodigoAlumno": "ALU-TEST",  
  "nombre": "Jorge",  
  "password": "1234",  
  "horasTotales": 0.0,  
  "fkCentro": "CEN01" }
```

PASO 2: Modificación de los datos creados (PUT) Cambiamos al alumno de empresa y actualizamos su email.

URL: `http://localhost:8088/api/alumnos/ALU-TEST`

Método: PUT

Body (JSON):

```
{ "idCodigoAlumno": "ALU-TEST",  
  "nombre": "Jorge",  
  "apellidos": "Pruebas Finales",  
  "email": "jorge.actualizado@gmail.com",  
  "fkCentro": "CEN01",  
  "fkEmpresa": "CIF002",  
  "fkProfesor": "DOC002",  
  "fkTutor": "TUT002" }
```

Resultado esperado:

```
{ "idCodigoAlumno": "ALU-TEST",  
  "email": "jorge.actualizado@gmail.com",  
  "fkEmpresa": "CIF002",  
  "fkTutor": "TUT002" }
```

PASO 3: Limpieza del sistema (DELETE)

Eliminamos el registro de prueba para dejar la base de datos como estaba.

URL: <http://localhost:8088/api/alumnos/ALU-TEST>

Método: DELETE

Resultado esperado: Status 204 No Content.